

Software Design Document (SDD)

Project: Gamified Platform (Backend) **Version:** 1.0.0 **Date:** April 2026

=====

1. Introduction

1.1 Purpose

The purpose of this document is to detail the architectural and software structural design of the Gamified Platform's backend API. It serves as a comprehensive reference for developers, engineers, and stakeholders to understand the core technological decisions, data models, integration points, and subsystem components.

1.2 Scope

This document covers the monolithic Node.js backend system, its interactions with the primary datastore (MongoDB), asynchronous background job processing (Agenda), external API integrations (OpenAI, Resend, Twilio, Google Auth, OneSignal), and generic API implementation standards.

=====

2. System Overview

The backend application serves as the centralized orchestration layer for a gamified educational experience. It manages user progressions through hierarchical curriculums while dynamically utilizing OpenAI to act as a personalized tutoring agent. The platform leverages psychological gamification hooks such as streaks, experience points (XP), unlockable lessons, virtual economies (coins/diamonds), and leaderboards to drive learner retention.

=====

3. Technology Stack & Architecture

3.1 Stack Choices

- **Runtime Environment:** Node.js (v20+)
- **Language:** TypeScript (Strict Mode)
- **Web Framework:** Express.js (v5)
- **Database Engine:** MongoDB
- **ODM (Object-Document Mapper):** Mongoose
- **Input Validation:** Zod
- **Logging:** Pino (Structured JSON logging)

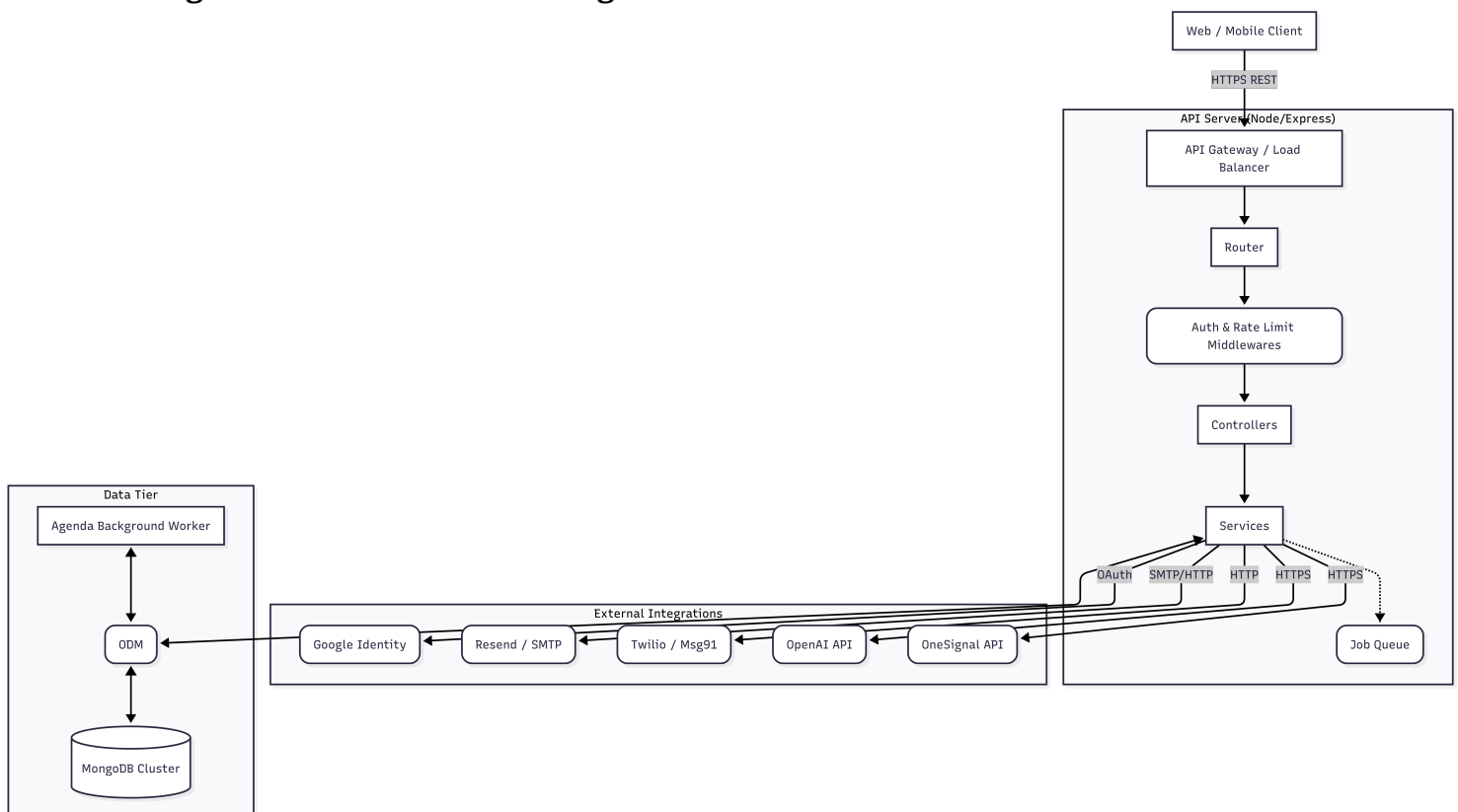
- **Job Queue Driver:** Agenda (backed by MongoDB)

3.2 Architectural Pattern

The application leverages a classic **Layered Monolithic Architecture**. Traffic routed to the Express application traverses the following execution lifecycle:

1. **Transport Layer (Express/Routes):** Inbound HTTP requests mapped to resources.
2. **Middlewares:** Authentication validation, rate-limiting, request-id assignment, payload parsing.
3. **Controllers:** Extraction of URL/Body parameters, Zod parsing/validation, invoking services, formatting output payloads using standard API Response classes.
4. **Service Layer:** Reusable domain/business logic functions (e.g., specific XP calculation strategies, AI prompting logic).
5. **Data Access Layer (Models):** Strict schema definition and lifecycle hooks handled by Mongoose.

3.3 High-Level Architecture Diagram



4. Subsystem Components Design

4.1 Authentication & Authorization

Uses robust short-lived JWTs (typically 15 min TTL) representing active authorization, accompanied by long-lived Opaque Refresh Tokens securely written to { httpOnly: true, secure: true } cookies and database entities.

- **Supported Mechanisms:** Direct OTP (Mobile/Email), Google OAuth 2.0 Identity verification.
- **Entities:** User, Otp, RefreshToken.
- **Role-Based Access Control (RBAC):** Middleware checks for learner, admin, and superadmin bounds (e.g., requireAdmin, requireSuperAdmin).

4.2 Curriculum Mapping Engine

Stores academic data strictly dimensionally to aid deep analytics and cohort structuring.

- **Hierarchy Flow:** Standard (e.g., CBSE 8th Grade) -- Subject (e.g., Science) -- Unit (e.g., Physics Fundamentals) -- Chapter (e.g., Motion) -- Lesson -- Quiz.
- Soft-delete logic exists allowing Admin users to restore accidentally wiped academic entities to maintain historical data integrities on past User query attempts.

4.3 Gamification Engine & Wallet

Centralizes the logic to quantify progress into rewards. Handled primarily through gamification.service.ts.

- **XP Strategy:** Based on difficulty curves attached to the quizzes and performance matching. Capped daily to prevent artificial binge-grinding.
- **Currency:** Employs dual currencies (coins, diamonds). Administered exclusively by system APIs via appending double-entry ledgers inside the WalletTransaction collection to create an immutable audit trail.
- **Streaks:** Increment loops based strictly on the user's localized lastActiveDate.

4.4 Artificial Intelligence Tutoring

Leverages prompt-engineering against OpenAI's models (chat.controller.ts & ai.service.ts).

- **Context Ingestion:** The system explicitly pulls the user's three lowest performing recent lesson attempts and injects the topics seamlessly into the hidden System Prompt. This implicitly tailors the AI's instruction behavior towards the user's blindspots without manual targeting.

- **Abuse Prevention:** Rate-limited explicitly by AI_DAILY_LIMIT via UserDailyUsage collections. Heuristically checks strings for cheating keywords (“give me answers”, “answer key”) to block direct plagiarism assistance.

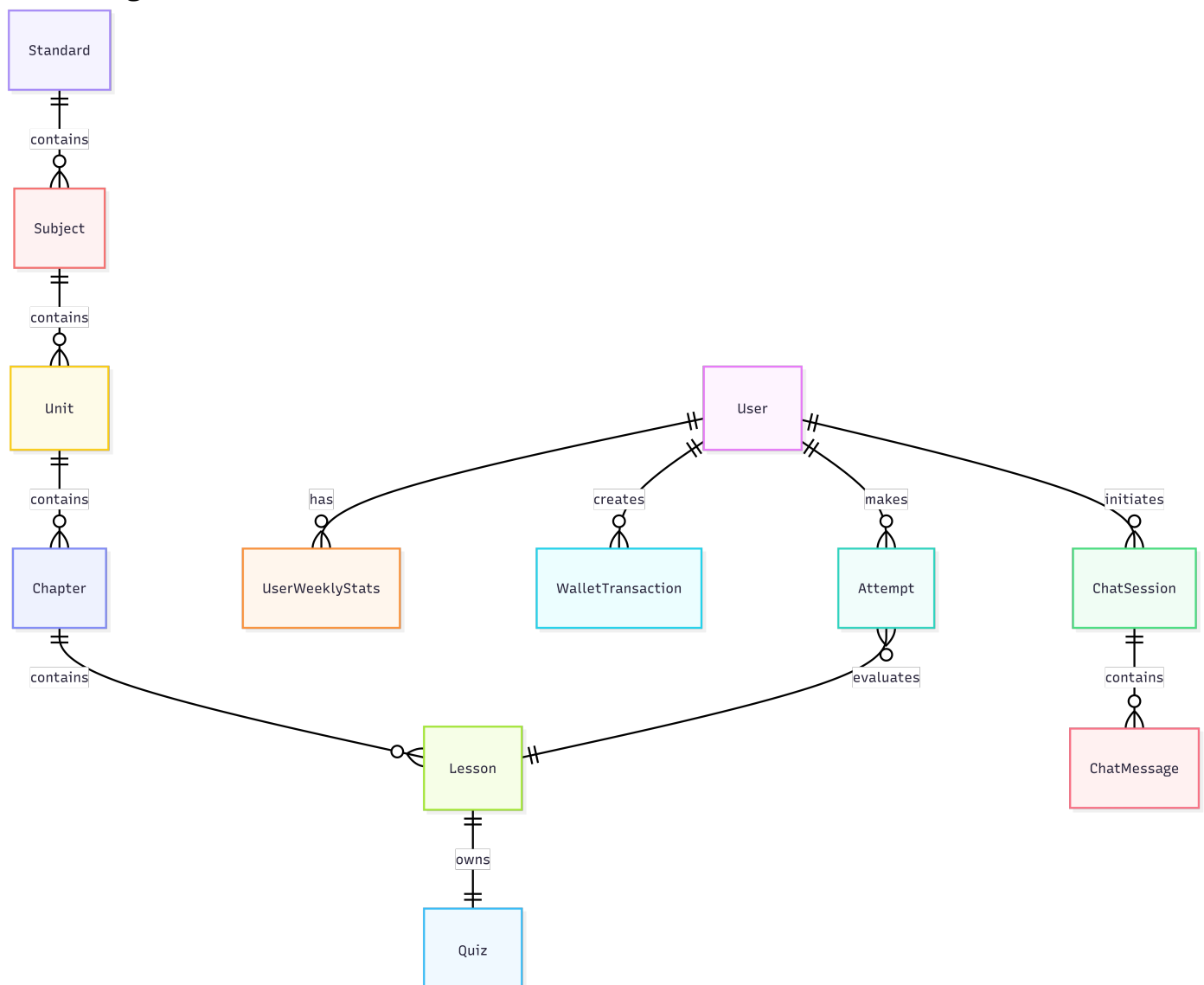
4.5 Background Processing (Workers)

Employs worker.ts as an out-of-band thread to decouple heavy computations from blocking the Event Loop on web servers.

- **Technology:** agenda operating via a MongoDB connection leveraging pessimistic locking.
- **Workloads:** Executing massive queries spanning thousands of users for weekly leaderboard recalibrations (RECOMPUTE_WEEKLY_LEADERBOARD).

5. Data Model Design

5.1 High-Level ERD



5.2 Key Entity Properties

- **User:** Core Identity. Extensible generic profile object schemas. Maintains raw denormalized aggregated counters (totalXP, level, streakCount).
- **Attempt:** Heavily transactional. Contains a snapshot array of the answered indices, the time spent, generated idempotent hashes to block duplicate requests across spotty connections, and the XP/Coins awarded explicitly for this instance.

=====

6. Communication Strategies & API Guidelines

6.1 Consistent Response Formatting

The backend mandates strict homogenous JSON structure definitions for successful (ok()) and failing (fail()) HTTP transactions to heavily simplify parsing in frontend clients (e.g., React, Flutter).

Success Pattern:

```
{
  "success": true,
  "data": { ... } // Varies by payload
}
```

Failure Pattern:

```
{
  "success": false,
  "errorCode": "VALIDATION_FAILED", // Machine parsable enumerations
  "message": "Human readable reason",
  "details": { ... } // Zod array schemas if syntax failed
}
```

6.2 Idempotency Strategy

Endpoints that result in user credit/monetary transactions (POST /attempts/submit) leverage idempotencyKey strings supplied organically by the frontend. The Attempt collection has a unique compound index holding the userId + idempotencyKey enforcing a hard block on duplicated XP/Coin rewards occurring during a network retry burst.

=====

7. Security Architecture

1. **DDoS & Abuse Prevention:** Global generic limiter enforced via express-rate-limit. Isolated, heavily restrictive tracking on intensive AI endpoint requests.
2. **Transport Enforcement:** helmet used to strip Express fingerprinting, establish stringent Content-Security-Policies, and force HTTPS HSTS where application layer proxies exist.
3. **Audits:** Any operation tagged under /v1/admin/* that forces a mutation intercepts payload values and archives an immutable entry inside AdminAuditLog detailing the actor, time, payload delta, and entity targeted.

NOTE

This baseline SDD encapsulates the current backend implementation as of v1.0.